CSE 554 Machine Learning Systems

Lecture 4 – Transformer Architecture

Baris Kasikci

BE BOUNDLESS

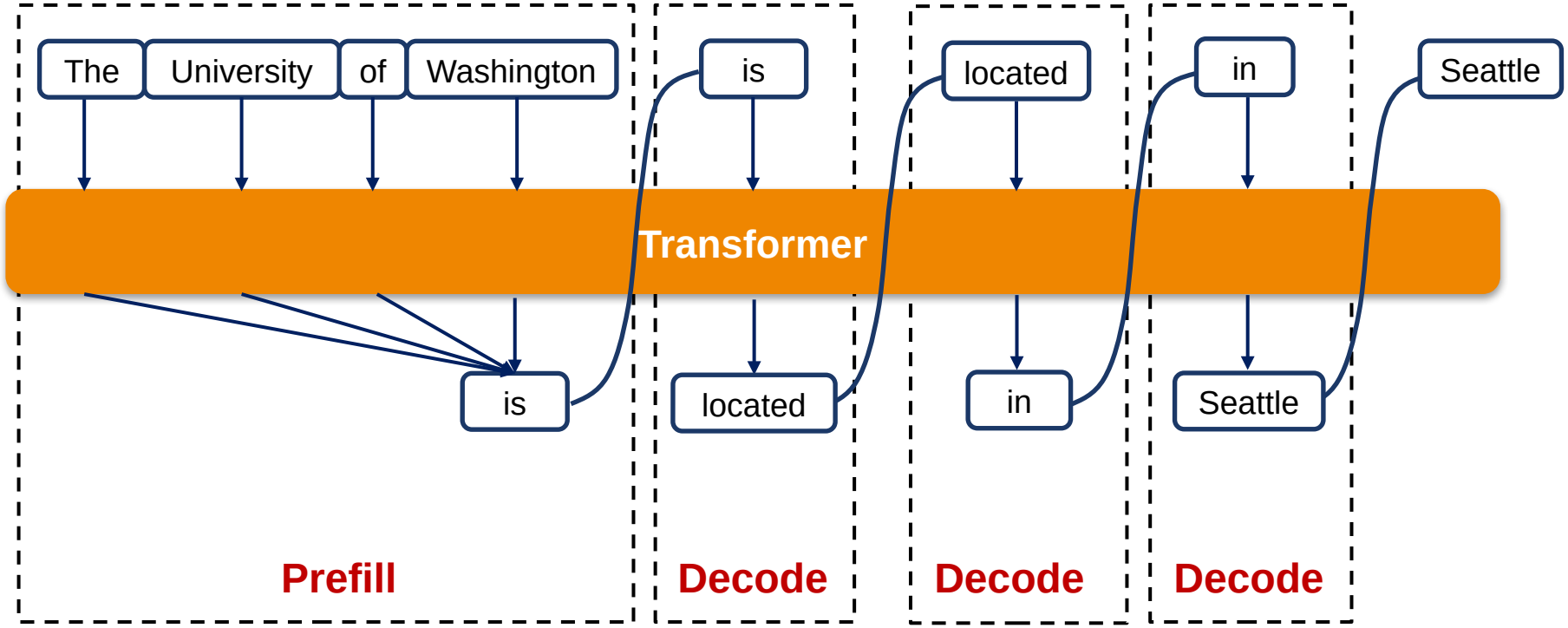
Class Overview

1. GPU Architecture and Programming
2. **Transformers (Architecture and Implementation)**
3. Performance modeling
4. Quantization and Sparsity
5. Memory Management
6. Mixture of Experts (MoE)
7. Batching
8. Speculative decoding
9. Parallelism
10. End-to-end serving
11. Agentic systems
12. Large-scale deployment

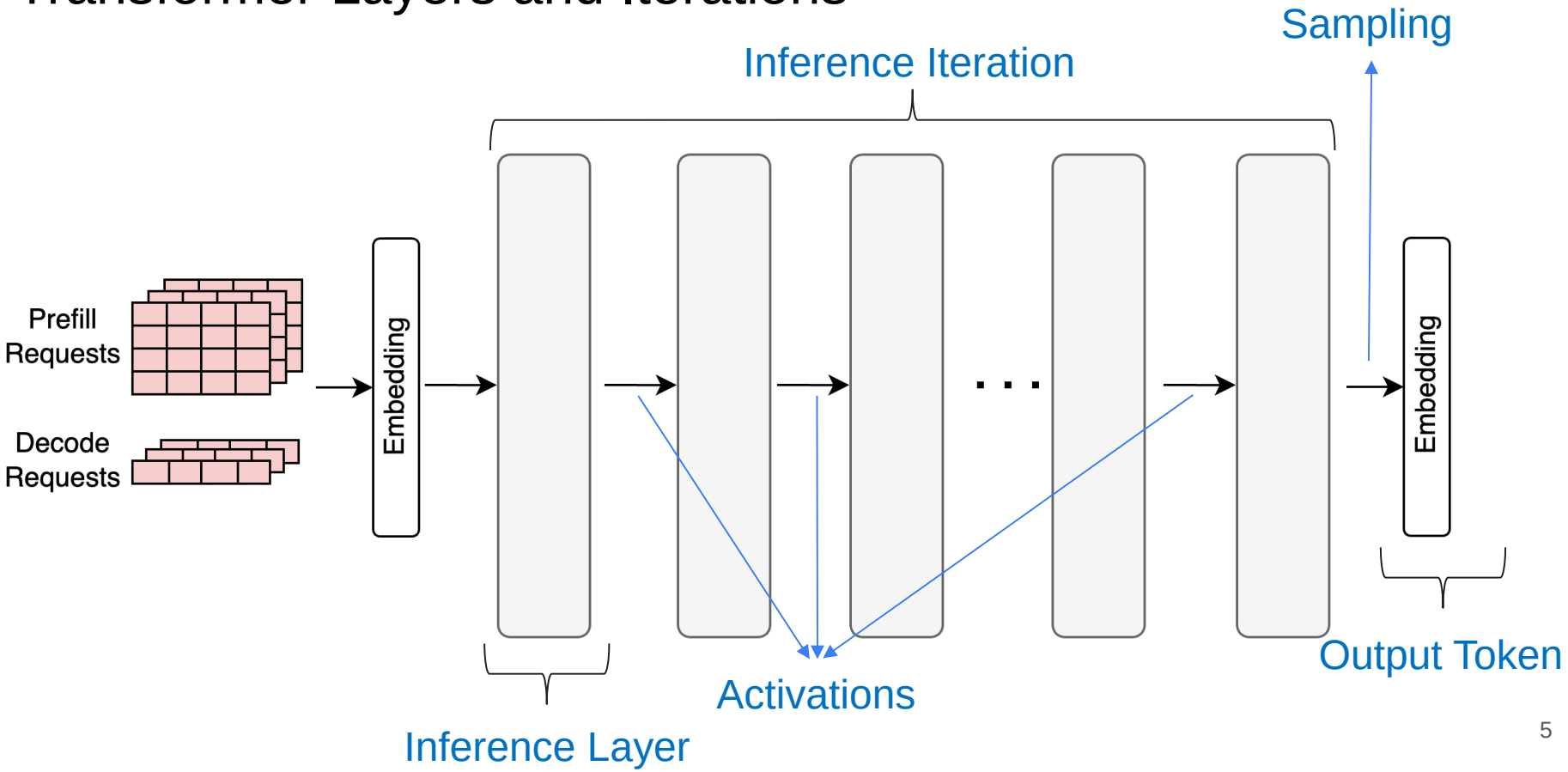
Today

- Transformer Architecture and Analytical Model

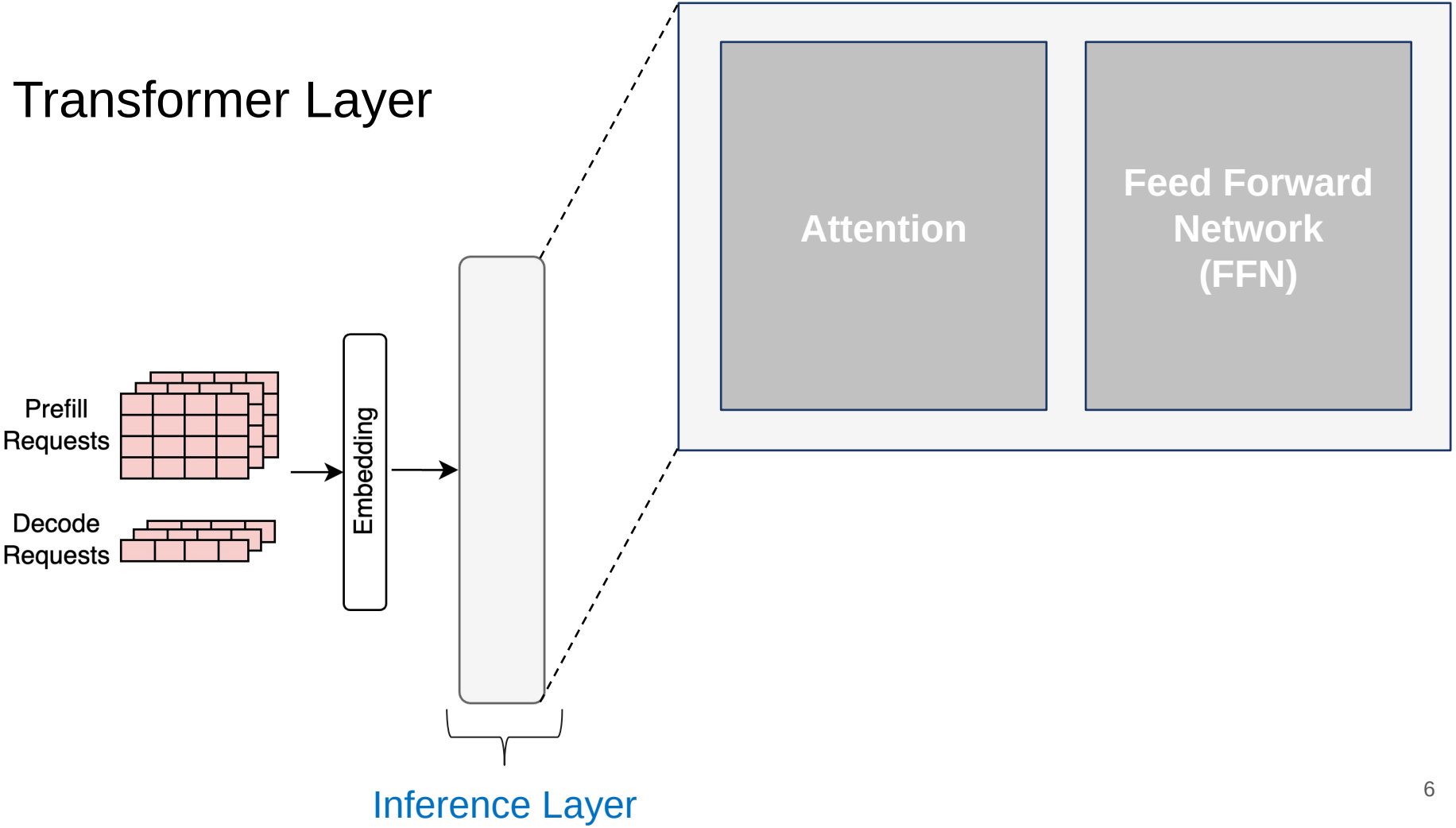
Prefill and Decode Phases of the Transformer



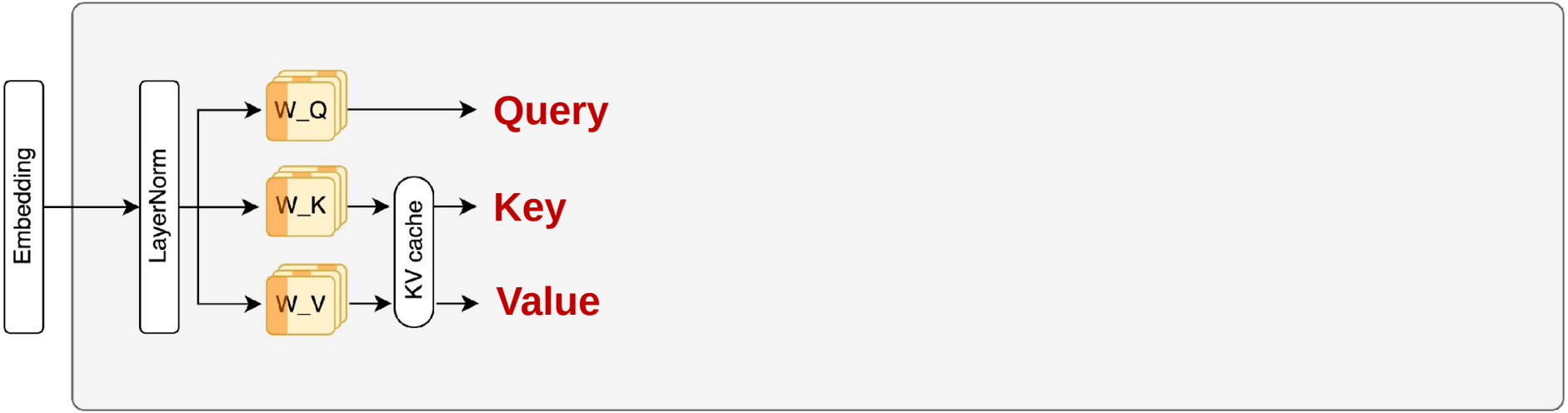
Transformer Layers and Iterations

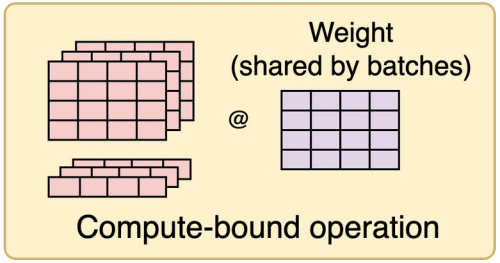


Transformer Layer



Decoder-Only Transformer* – QKV Generation





*Llama architecture

Keys, Queries, Values[1]

Query : The thing the transformer is looking for
(Is there an adjective before me?)

W_Q

Input: A blue creature roamed the verdant forest

W_K

W_V

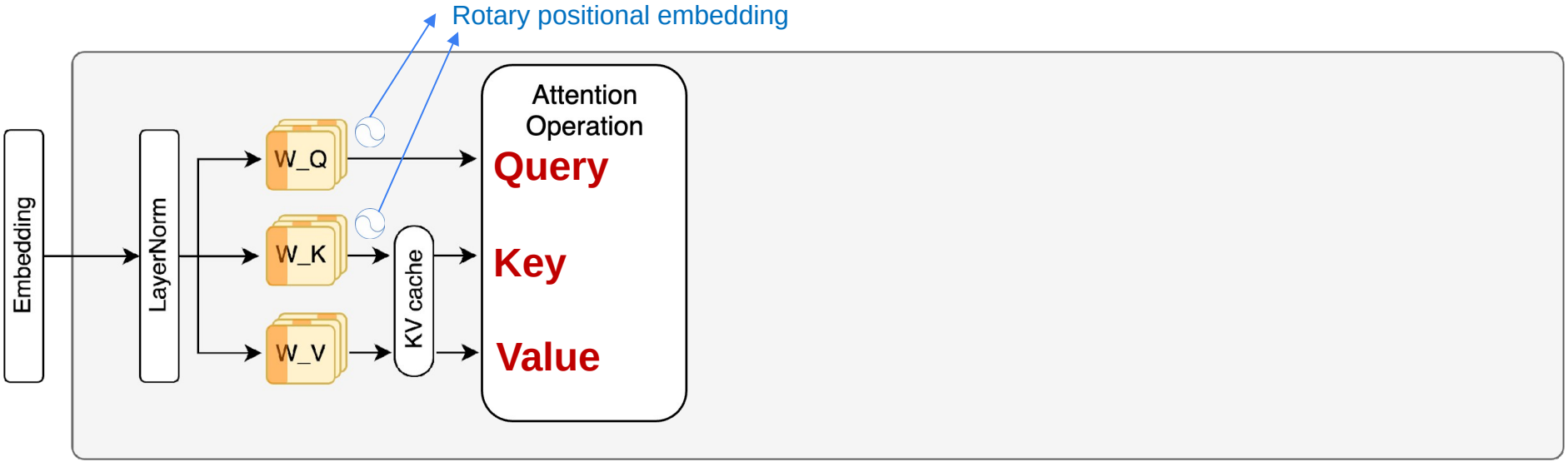
Key : What's available
(Yes, there is, and I am here)

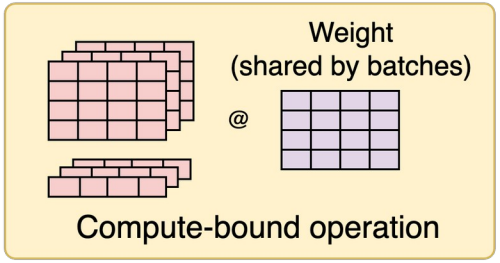
Value : What needs to be updated
to assimilate context

"Creature" "Blue Creature"

[1] https://www.youtube.com/watch?v=eMlx5fFNoYc

Decoder-Only Transformer – QKV Generation





Interesting new research on the harms of positional embeddings: https://pub.sakana.ai/DroPE/

Self-Attention

$$\underbrace{(\ \text{Query}\ \ \text{Key}^{T}\ )}_{\text{Attention Score}}\ \text{Value}$$

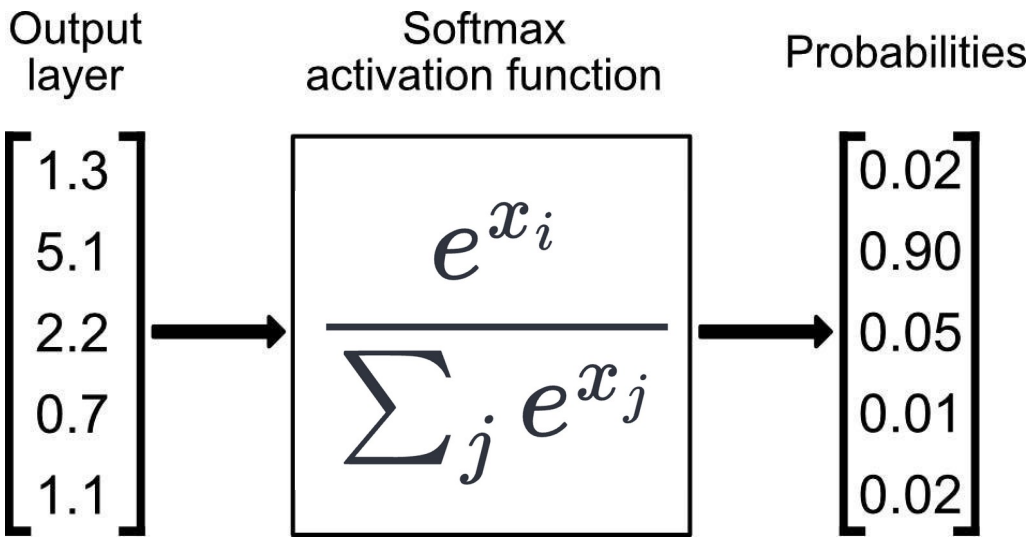
(Measures how well the queries and keys align)

Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{\text{Query}\ \ \text{Key}^T}{\sqrt{d_k}}\right)\text{Value}$$

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Softmax



https://towardsdatascience.com/softmax-activation-function-explained-a7e1bc3ad60/

Causal Self-Attention

A blue creature roamed the verdant forest

A → **?**

A blue → **?**

A blue creature → **?**

A blue creature roamed → **?**

A blue creature roamed the → **?**

A blue creature roamed the verdant → **?**

Causal Self-Attention

$$\vec{K_i} = \vec{E_i} \quad \overrightarrow{W_K_i}$$

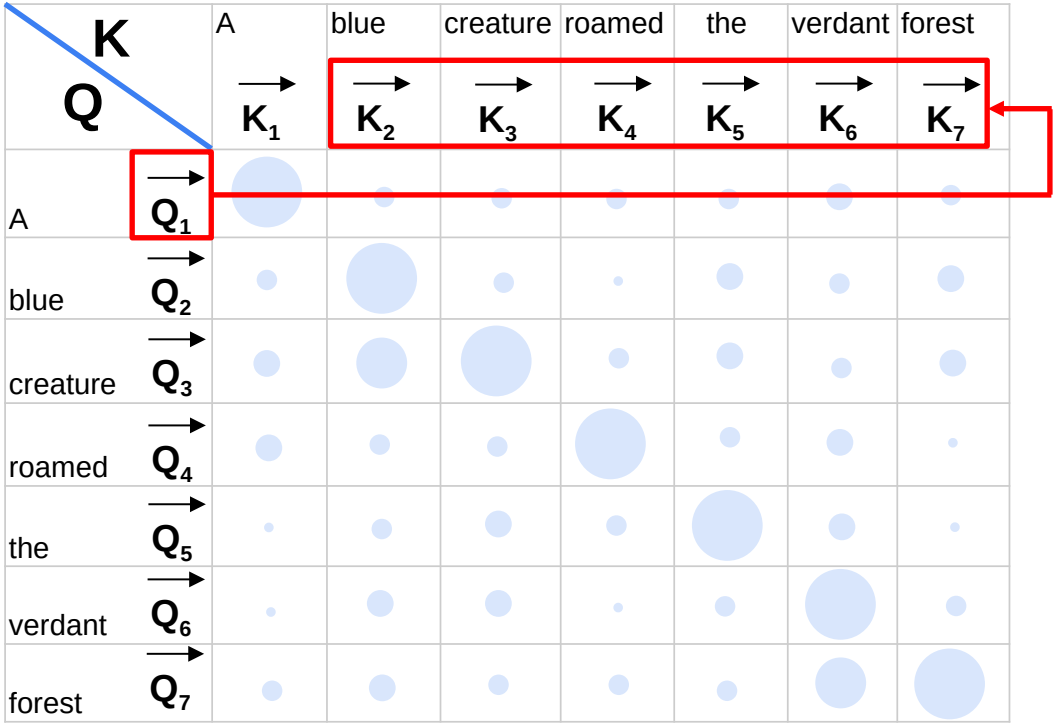
$$\vec{Q_i} = \vec{E_i} \quad \overrightarrow{W_Q_i}$$

K / Q	A $\vec{K_1}$	blue $\vec{K_2}$	creature $\vec{K_3}$	roamed $\vec{K_4}$	the $\vec{K_5}$	verdant $\vec{K_6}$	forest $\vec{K_7}$
A $\vec{Q_1}$	●	•	•	•	•	•	•
blue $\vec{Q_2}$	•	●	•	·	•	•	•
creature $\vec{Q_3}$	•	•	●	•	•	•	•
roamed $\vec{Q_4}$	•	•	•	●	•	•	·
the $\vec{Q_5}$	·	•	•	•	●	•	·
verdant $\vec{Q_6}$	·	•	•	·	•	●	•
forest $\vec{Q_7}$	•	•	•	•	•	•	●

Causal Self-Attention

$$\vec{K_i} = \vec{E_i} \quad \overrightarrow{W_K_i}$$

$$\vec{Q_i} = \vec{E_i} \quad \overrightarrow{W_Q_i}$$



Tokens should not attend to future tokens:
- Otherwise, the neural network can memorize

to predict the next token

Causal Self-Attention

$\vec{K_i} = \vec{E_i} \quad \vec{W_K_i}$

$\vec{Q_i} = \vec{E_i} \quad \vec{W_Q_i}$

K / Q	A $\vec{K_1}$	blue $\vec{K_2}$	creature $\vec{K_3}$	roamed $\vec{K_4}$	the $\vec{K_5}$	verdant $\vec{K_6}$	forest $\vec{K_7}$
A $\vec{Q_1}$	●	-∞	-∞	-∞	-∞	-∞	-∞
blue $\vec{Q_2}$	●	●	-∞	-∞	-∞	-∞	-∞
creature $\vec{Q_3}$	●	●	●	-∞	-∞	-∞	-∞
roamed $\vec{Q_4}$	●	●	●	●	-∞	-∞	-∞
the $\vec{Q_5}$	●	●	●	●	●	-∞	-∞
verdant $\vec{Q_6}$	●	●	●	●	●	●	-∞
forest $\vec{Q_7}$	●	●	●	●	●	●	●

Causal mask

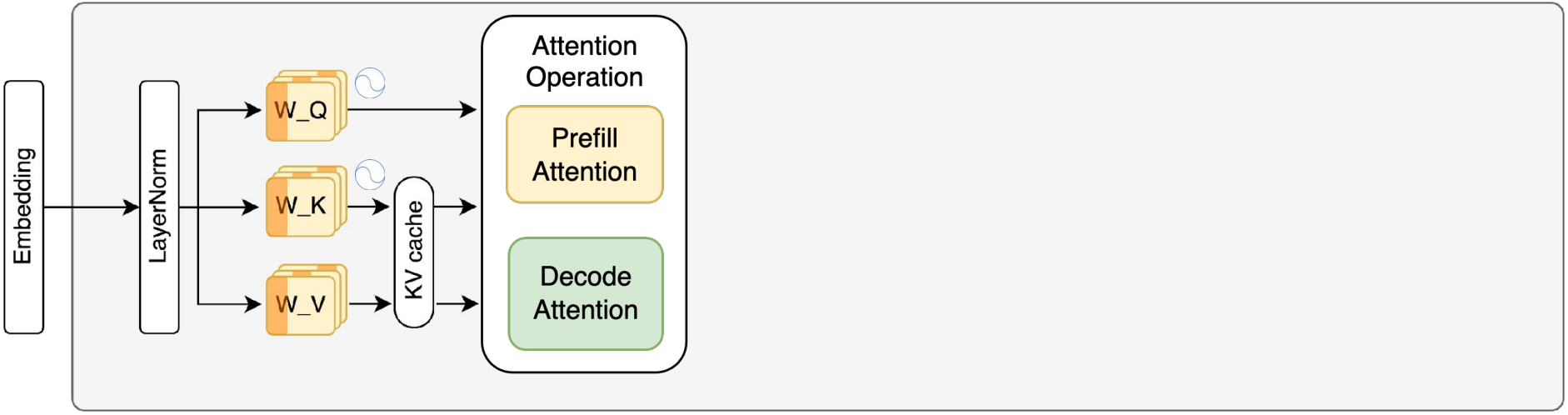
Attention (Q, K, V) =

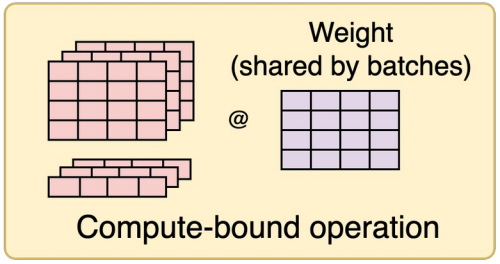
$$\text{softmax}\left(\frac{\text{Query}\ \text{Key}^T}{\sqrt{d_k}}\right)\text{Value}$$

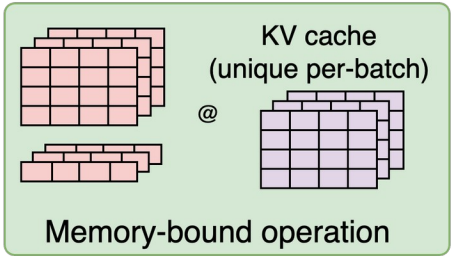
$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

xi → -∞, softmax(xi) → 0

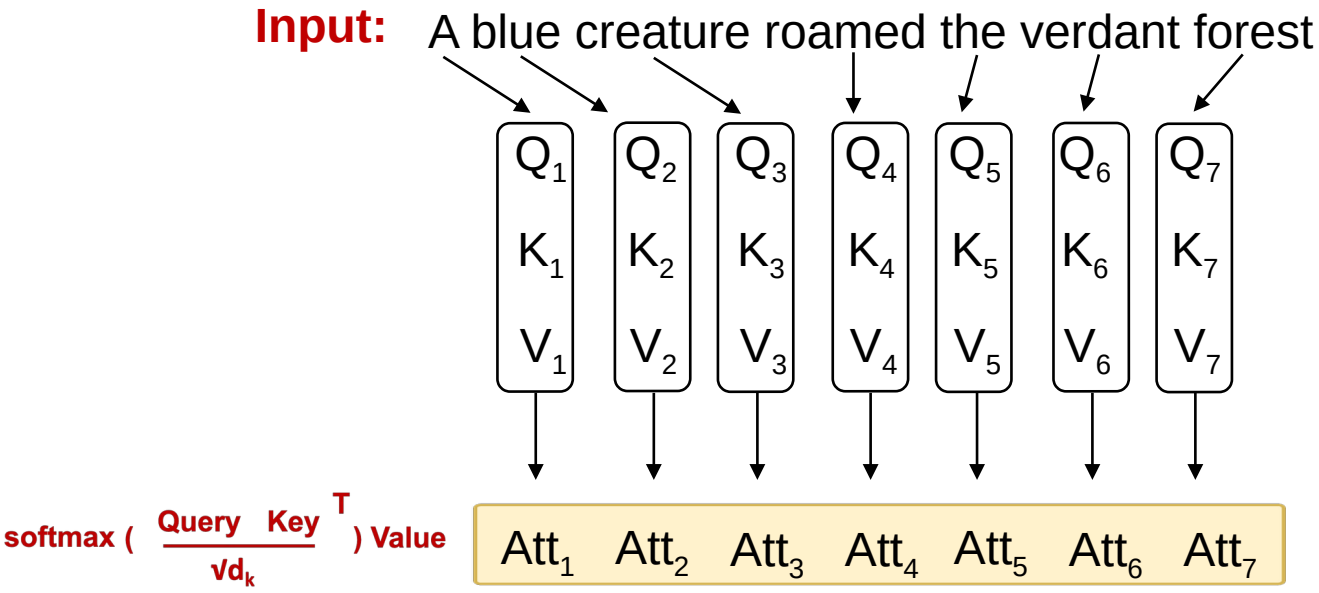
Decoder-Only Transformer – Prefill/Decode Attention







Prefill Attention



Compute-bound operation

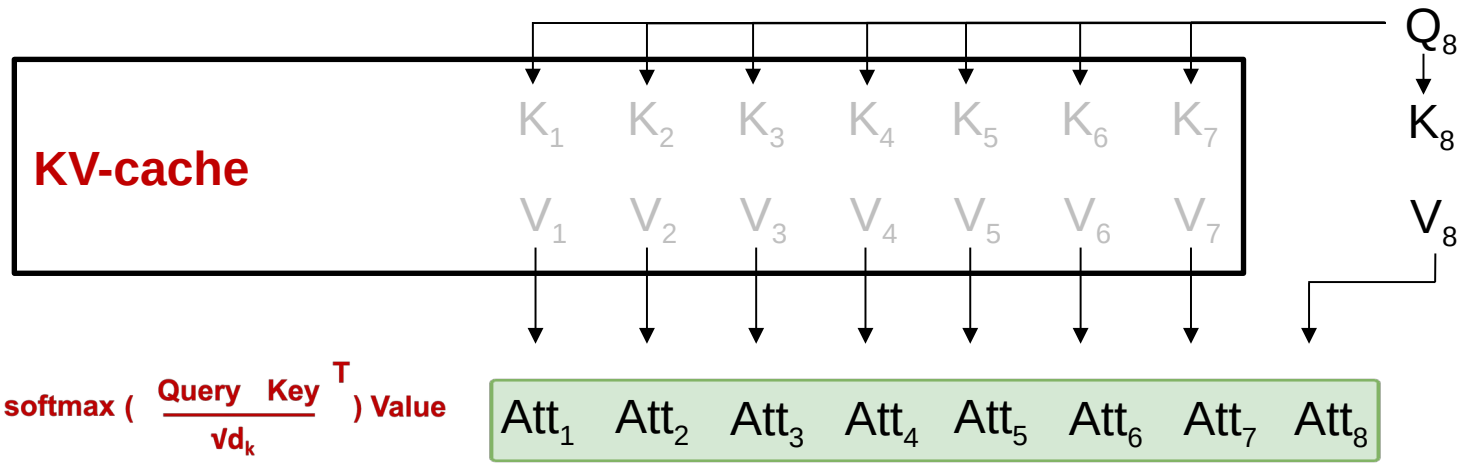
Decode Attention

Input: A blue creature roamed the verdant forest **<token$_8$>**

	Q_1	Q_2	Q_3	Q_4	Q_5	Q_6	Q_7	Q_8
KV-cache	K_1	K_2	K_3	K_4	K_5	K_6	K_7	K_8
	V_1	V_2	V_3	V_4	V_5	V_6	V_7	V_8

Decode Attention

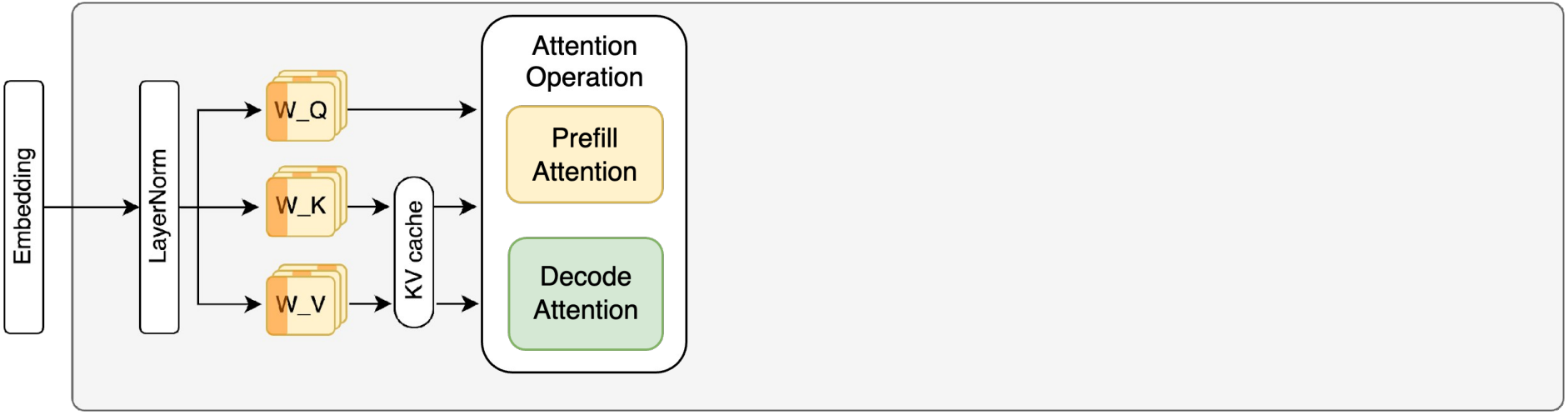
Input: A blue creature roamed the verdant forest **<token$_8$>**

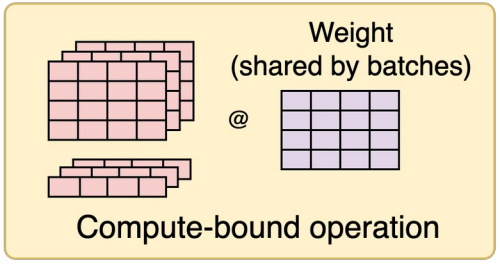


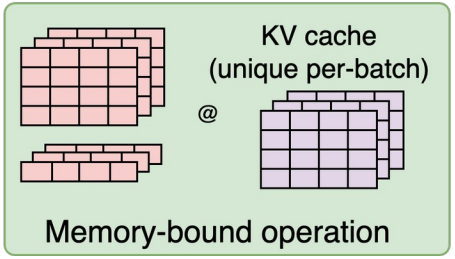
$$\text{softmax}\left(\frac{\text{Query}\ \text{Key}^T}{\sqrt{d_k}}\right)\text{Value}$$

Memory-bound operation

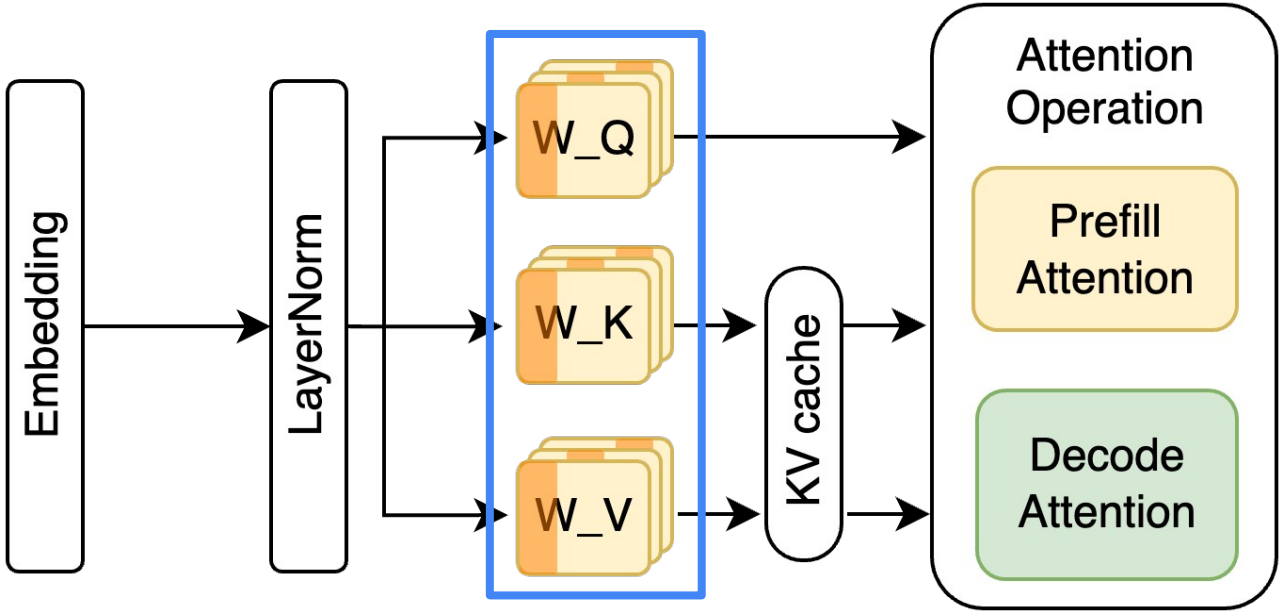
Decoder-Only Transformer – Prefill/Decode Attention



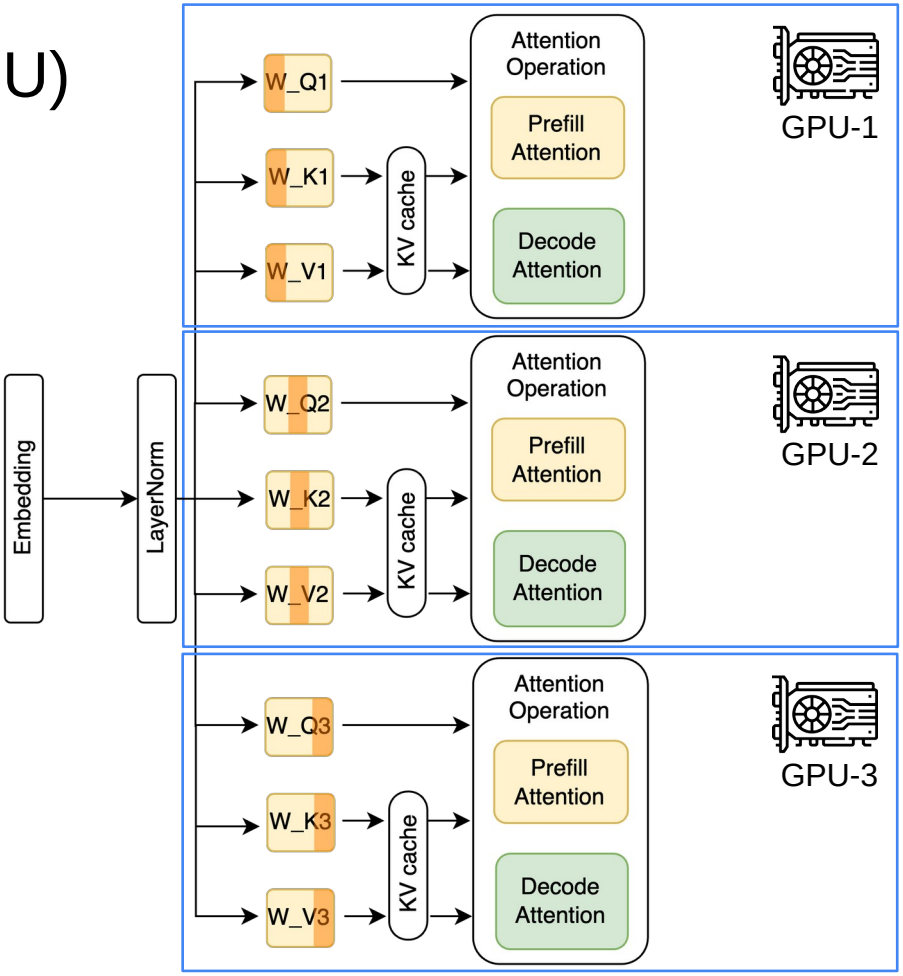




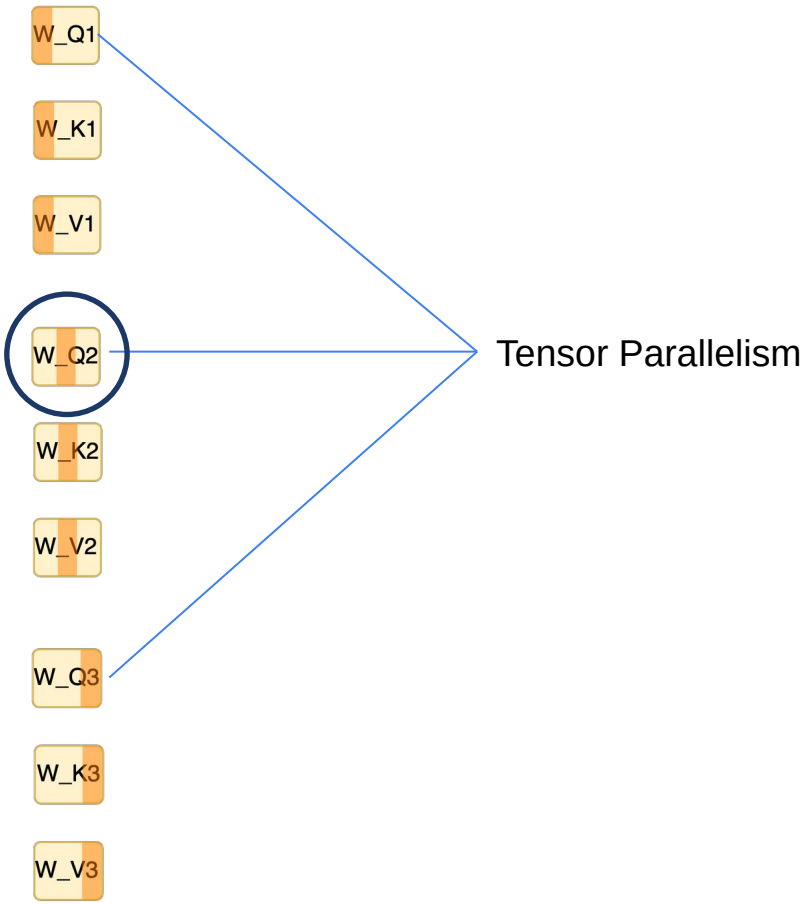
Attention


Embedding
LayerNorm
W_Q
W_K
W_V
KV cache
Attention Operation
Prefill Attention
Decode Attention

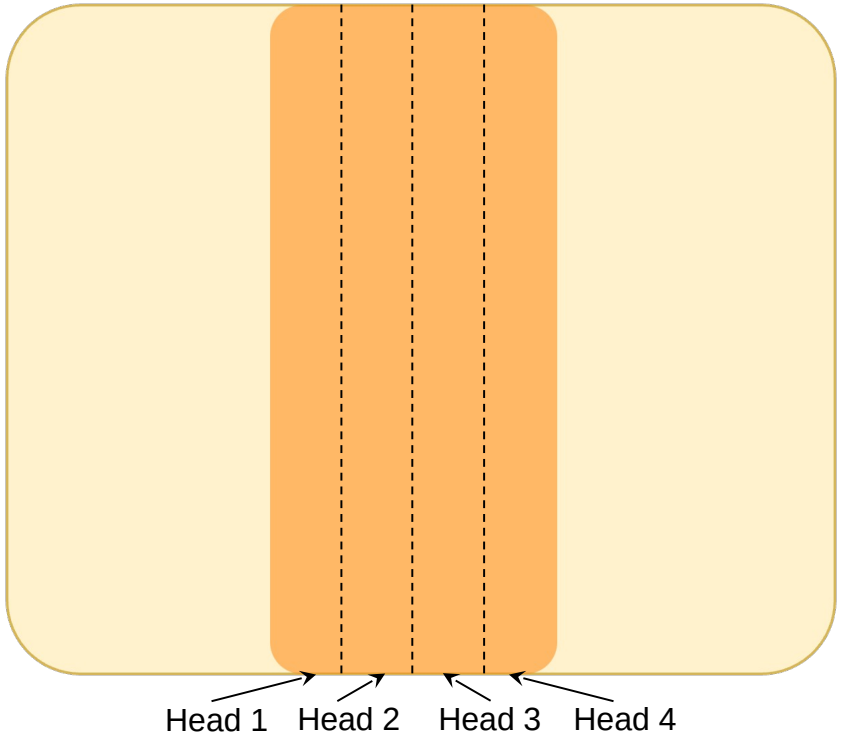
Attention (Multi-GPU)



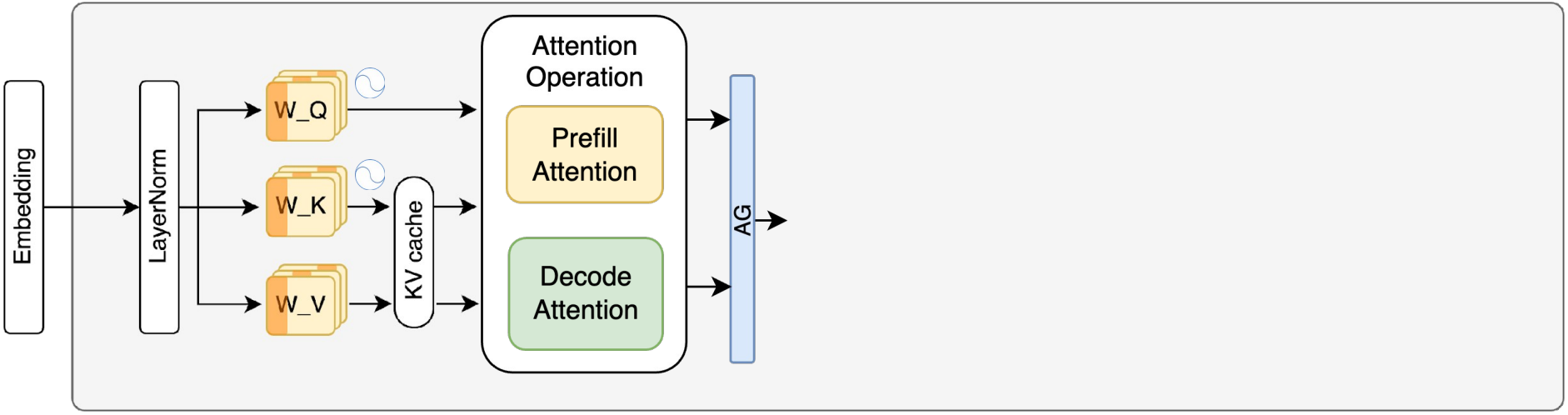
Attention (Multi-GPU)

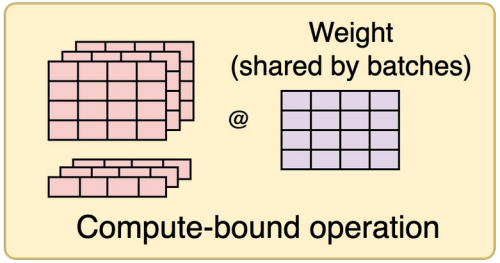


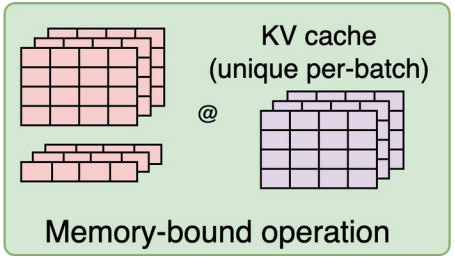
Multi-Head Attention (Multi-GPU)



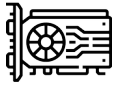
Decoder-Only Transformer – Attention AllGather





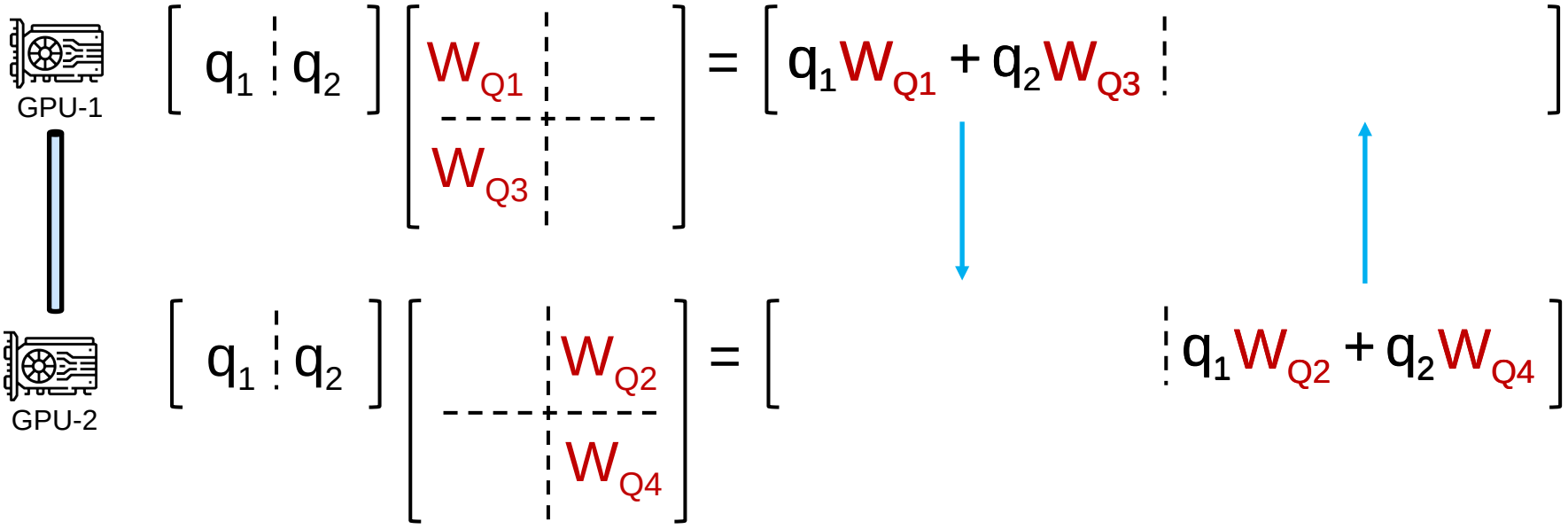


AllGather



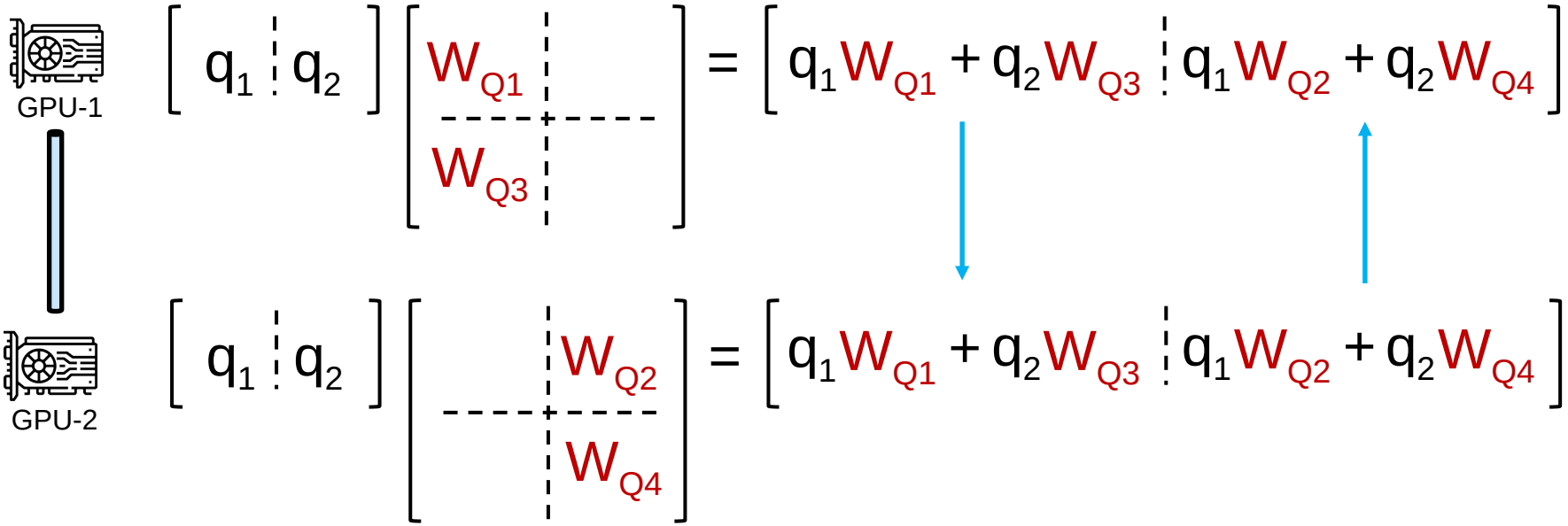
$$\begin{bmatrix} q_1 & q_2 \end{bmatrix} \begin{bmatrix} W_{Q1} & W_{Q2} \\ W_{Q3} & W_{Q4} \end{bmatrix} = \begin{bmatrix} q_1 W_{Q1} + q_2 W_{Q3} & q_1 W_{Q2} + q_2 W_{Q4} \end{bmatrix}$$

AllGather



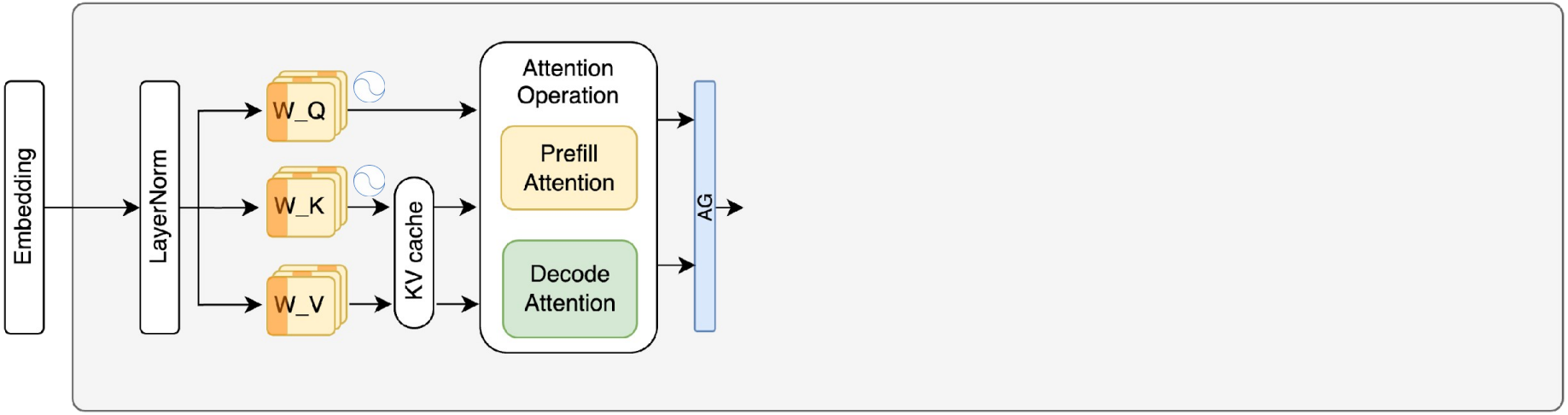
Network-bound operation

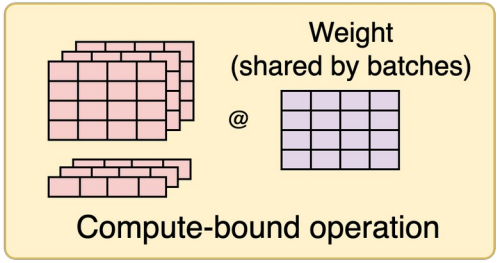
AllGather

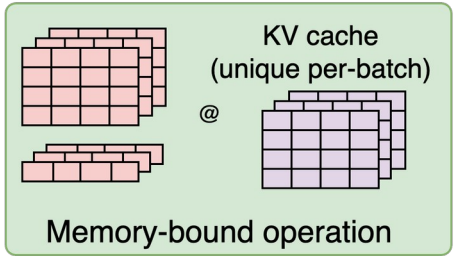


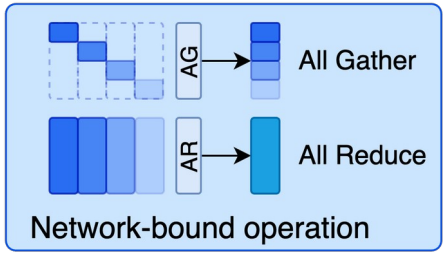
Network-bound operation

Decoder-Only Transformer – AllGather

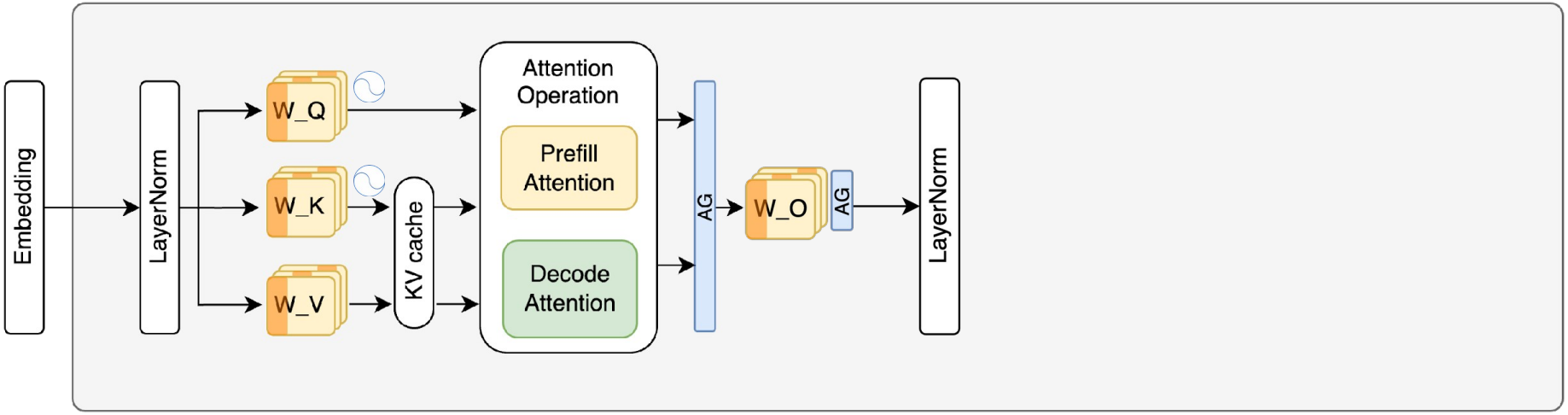


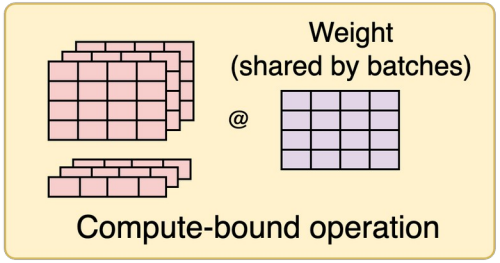


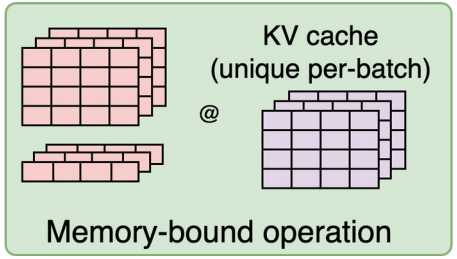


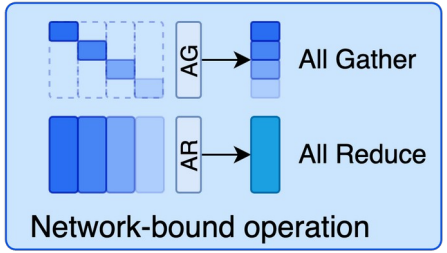


Decoder-Only Transformer – Output Projection & AllGather #2

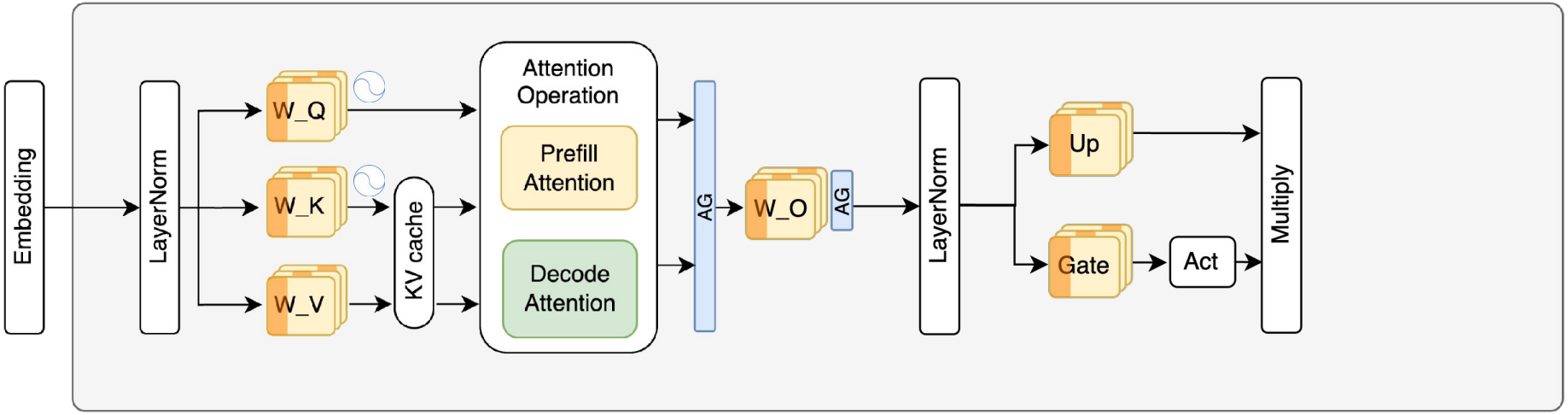


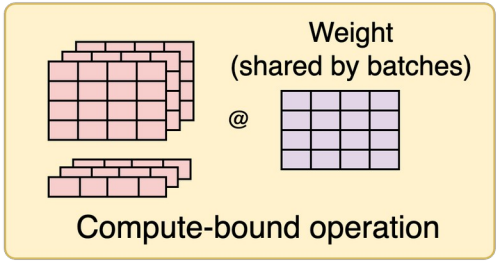


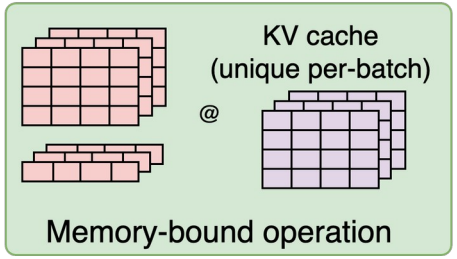


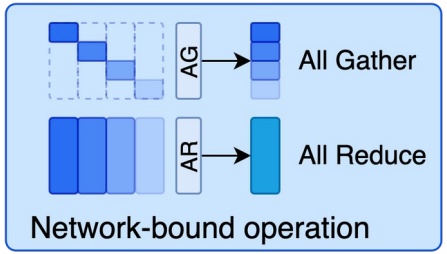


Decoder-Only Transformer – Feed Forward Network (FFN)

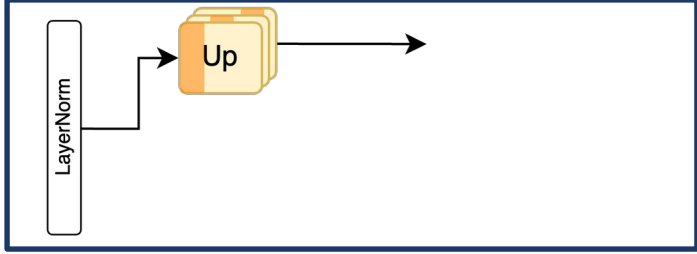








FFN - Up



GPU 1

$$\begin{bmatrix} o_1 & o_2 \end{bmatrix} \begin{bmatrix} W_{U1} & \\ W_{U3} & \end{bmatrix} = \begin{bmatrix} o_1 W_{U1} + o_2 W_{U3} & \end{bmatrix}$$

GPU 2

$$\begin{bmatrix} o_1 & o_2 \end{bmatrix} \begin{bmatrix} & W_{U2} \\ & W_{U4} \end{bmatrix} = \begin{bmatrix} & o_1 W_{U2} + o_2 W_{U4} \end{bmatrix}$$

FFN – Gate

GPU 1

$$\begin{bmatrix} o_1 & o_2 \end{bmatrix} \begin{bmatrix} W_{G1} & \\ W_{G3} & \end{bmatrix} = \begin{bmatrix} o_1 W_{G1} + o_2 W_{G3} & \end{bmatrix}$$

GPU 2

$$\begin{bmatrix} o_1 & o_2 \end{bmatrix} \begin{bmatrix} & W_{G2} \\ & W_{G4} \end{bmatrix} = \begin{bmatrix} & o_1 W_{G2} + o_2 W_{G4} \end{bmatrix}$$

FFN – Gate

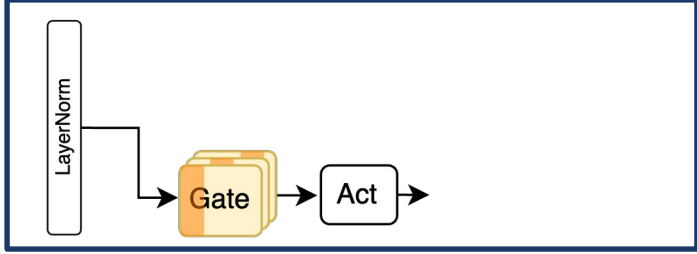
GPU 1

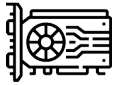
$$\left[\, o_1 W_{G1} + o_2 W_{G3} \;\vdots\; \right]$$

GPU 2

$$\left[\; \vdots\; o_1 W_{G2} + o_2 W_{G4} \,\right]$$

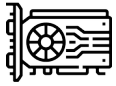
FFN – Gate & Act





GPU 1

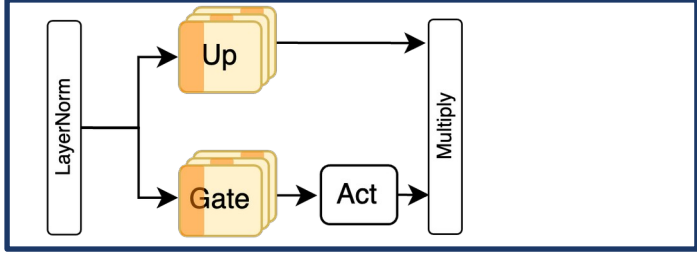
$$\sigma\left[\, o_1 W_{G1} + o_2 W_{G3} \;\vdots\; \right]$$



GPU 2

$$\sigma\left[\; \vdots\; o_1 W_{G2} + o_2 W_{G4} \,\right]$$

FFN – Multiplication



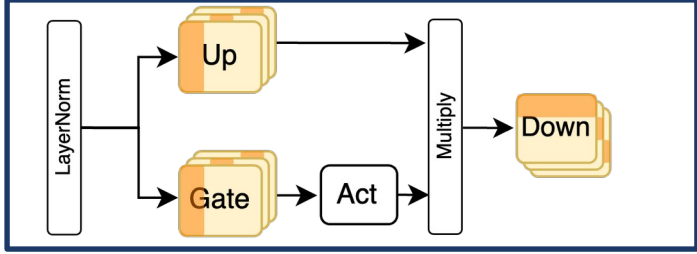
GPU 1

$$\left[\sigma\left(o_1 W_{G1} + o_2 W_{G3}\right)\right] \odot \left[o_1 W_{U1} + o_2 W_{U3}\right] = M_1$$

GPU 2

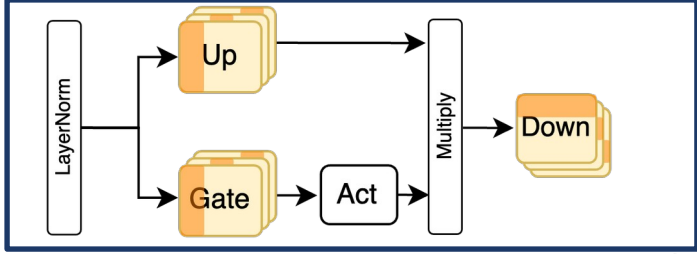
$$\left[\sigma\left(o_1 W_{G2} + o_2 W_{G4}\right)\right] \odot \left[o_1 W_{U2} + o_2 W_{U4}\right] = M_2$$

FFN – Down Projection



$$\begin{bmatrix} M_1 & M_2 \end{bmatrix} \left[\begin{array}{c:c} D_1 & D_2 \\ \hdashline D_3 & D_4 \end{array}\right] = \left[\begin{array}{c:c} M_1 D_1 + M_2 D_3 & M_1 D_2 + M_2 D_4 \end{array}\right]$$

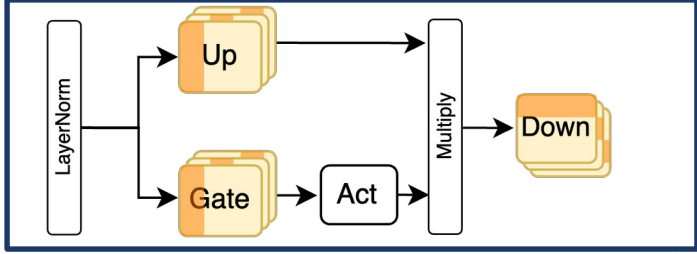
FFN – Down Projection



$$\begin{bmatrix} M_1 & M_2 \end{bmatrix} \left[\begin{array}{c:c} D_1 & D_2 \\ \hdashline D_3 & D_4 \end{array}\right]$$

$$\left[\begin{array}{c:c} M_1 D_1 + M_2 D_3 & M_1 D_2 + M_2 D_4 \end{array}\right]$$

FFN – Down Projection



$$\left[\begin{array}{c:c} M_1D_1 + M_2D_3 & M_1D_2 + M_2D_4 \end{array}\right]$$

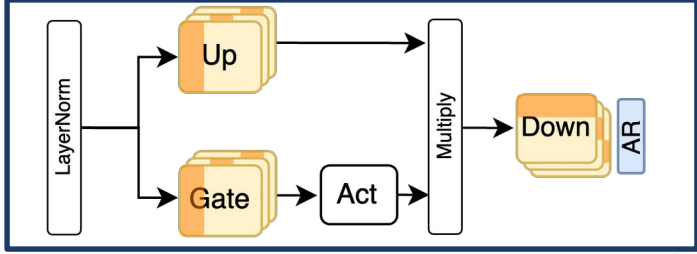
GPU 1

$$M_1 \begin{bmatrix} D_1 & D_2 \end{bmatrix} = \left[\begin{array}{c:c} M_1D_1 & M_1D_2 \end{array}\right]$$

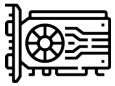
GPU 2

$$M_2 \begin{bmatrix} D_3 & D_4 \end{bmatrix} = \left[\begin{array}{c:c} M_2D_3 & M_2D_4 \end{array}\right]$$

FFN – AllReduce (ReduceScatter +

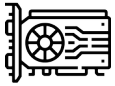


$$\left[\begin{array}{cc:cc} M_1D_1 & + & M_2D_3 & M_1D_2 & + & M_2D_4 \end{array}\right]$$



GPU 1

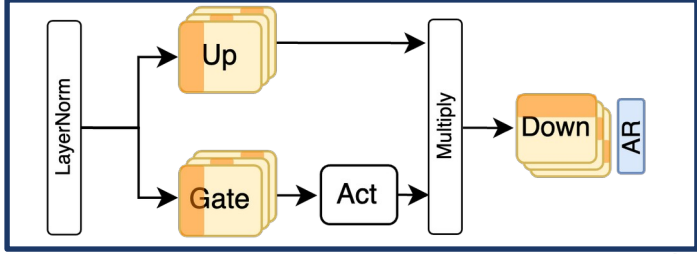
$$M_1 \begin{bmatrix} D_1 & D_2 \end{bmatrix} = \left[\begin{array}{cc:cc} M_1D_1 & & M_1D_2 & \end{array}\right]$$



GPU 2

$$M_2 \begin{bmatrix} D_3 & D_4 \end{bmatrix} = \left[\begin{array}{cc:cc} & M_2D_3 & & M_2D_4 \end{array}\right]$$

FFN – AllReduce (ReduceScatter + AllGather)



$$\left[\begin{array}{cc:cc} M_1D_1 & + & M_2D_3 & M_1D_2 + M_2D_4 \end{array}\right]$$

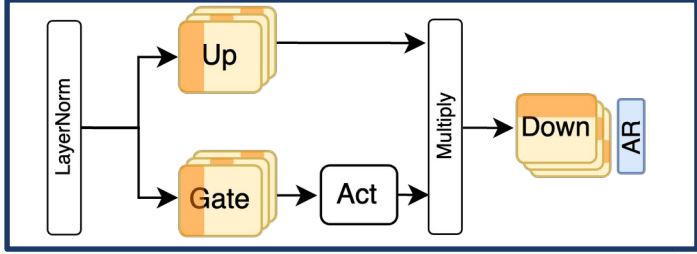
GPU 1

$$M_1 \begin{bmatrix} D_1 & D_2 \end{bmatrix} = \left[\begin{array}{cc:cc} M_1D_1 & + & M_2D_3 & \\ & & & \end{array}\right]$$

GPU 2

$$M_2 \begin{bmatrix} D_3 & D_4 \end{bmatrix} = \left[\begin{array}{cc:cc} & & M_1D_2 & + M_2D_4 \end{array}\right]$$

FFN – AllReduce (ReduceScatter + AllGather)



$$\left[\begin{array}{cc:cc} M_1D_1 & + & M_2D_3 & M_1D_2 & + & M_2D_4 \end{array}\right]$$

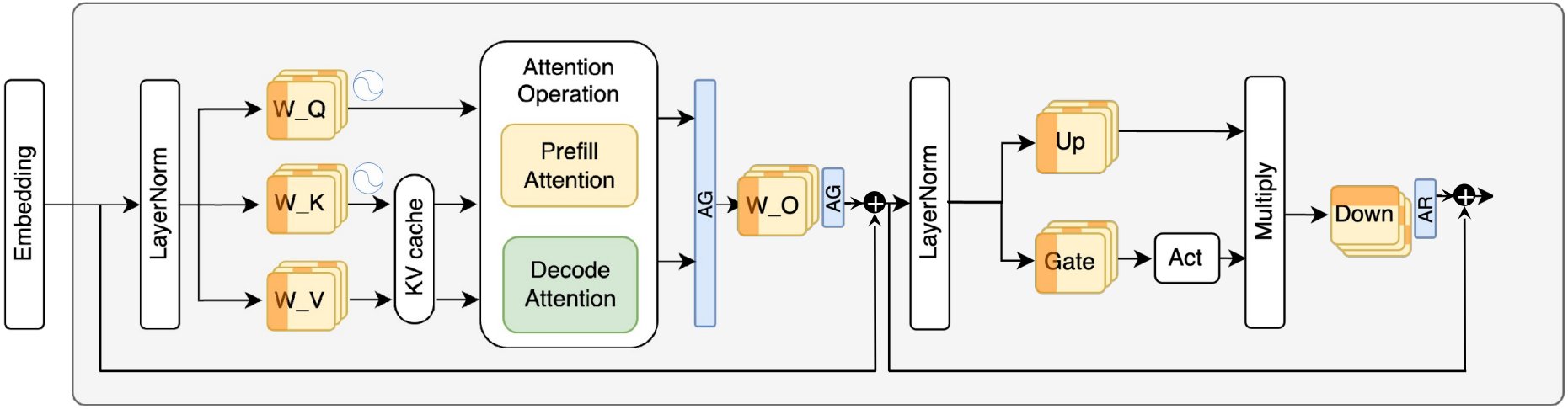
GPU 1

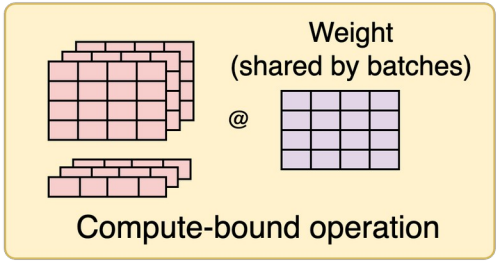
$$M_1 \begin{bmatrix} D_1 & D_2 \end{bmatrix} = \left[\begin{array}{cc:cc} M_1D_1 & + & M_2D_3 & M_1D_2 & + & M_2D_4 \end{array}\right]$$

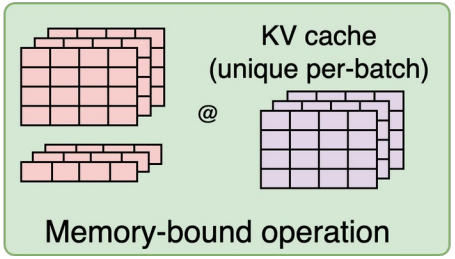
GPU 2

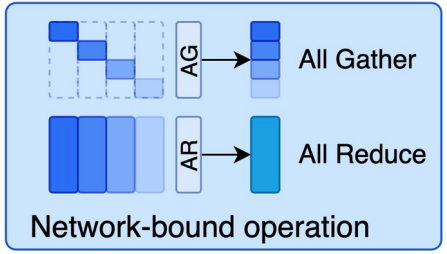
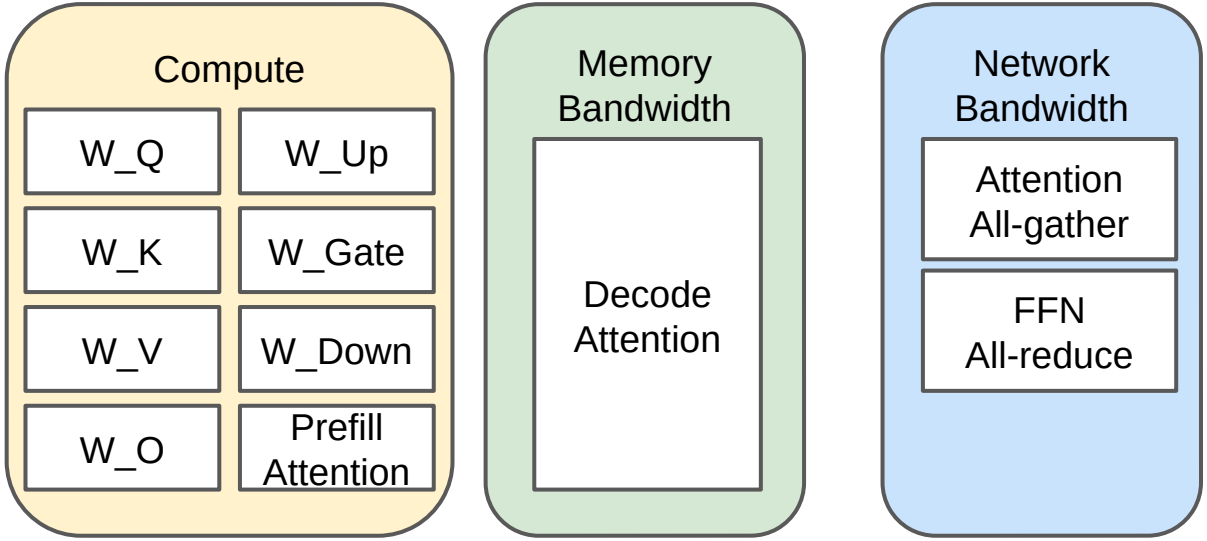
$$M_2 \begin{bmatrix} D_3 & D_4 \end{bmatrix} = \left[\begin{array}{cc:cc} M_1D_1 & + & M_2D_3 & M_1D_2 & + & M_2D_4 \end{array}\right]$$

Decoder-Only Transformer









Matching Operation Needs With GPU Resources